

void A (struct node *t)

{ if (t)

{ 1. Pf ("%c", t->data)

2. B (t->left)

3 B (t->right)

}

}

similar
to
preorder.

void B (struct node *t)

{ if (t)

{ 1. A (t->left)

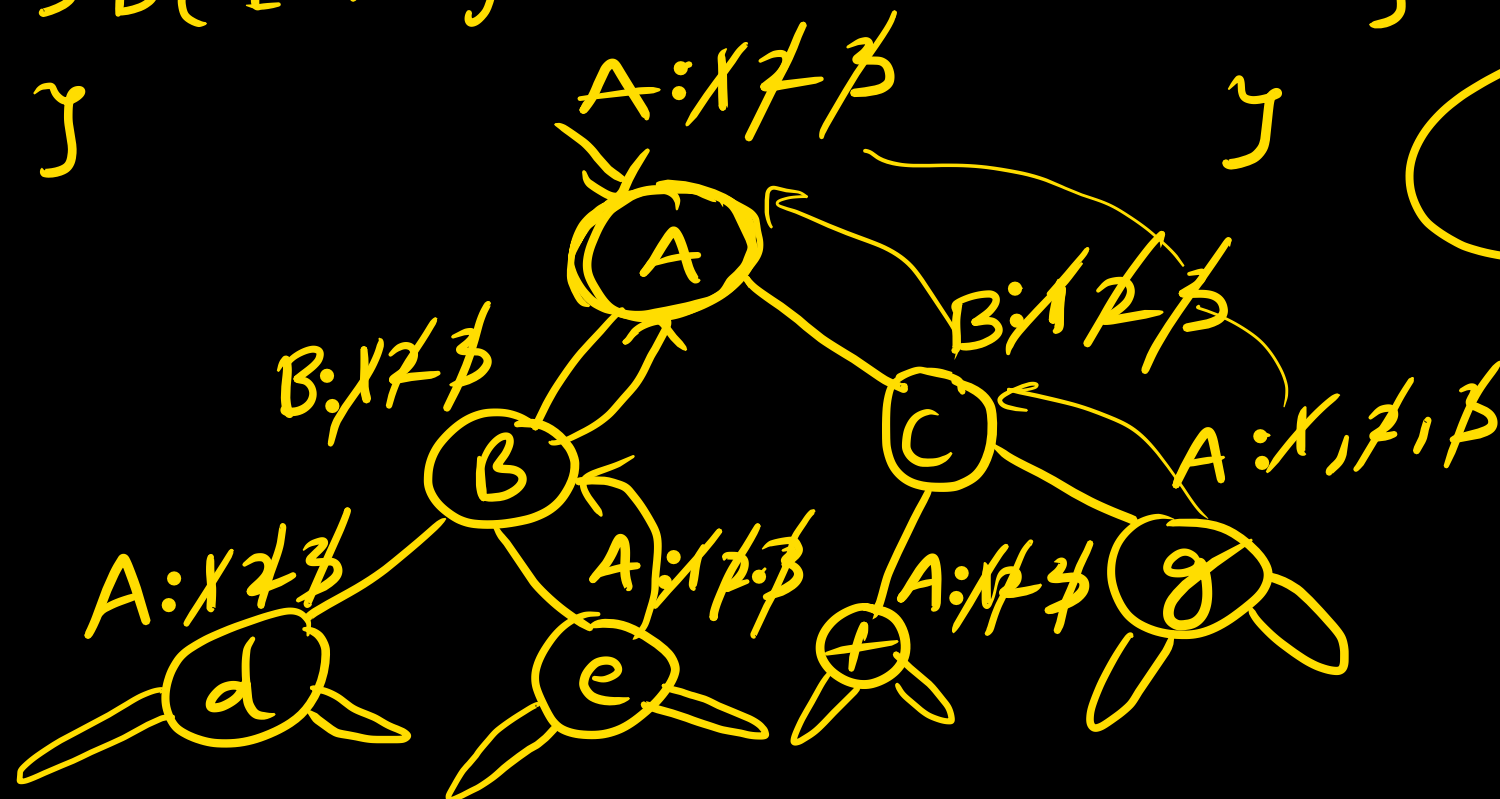
2. Pf ("%c", data)

3. A (t->right)

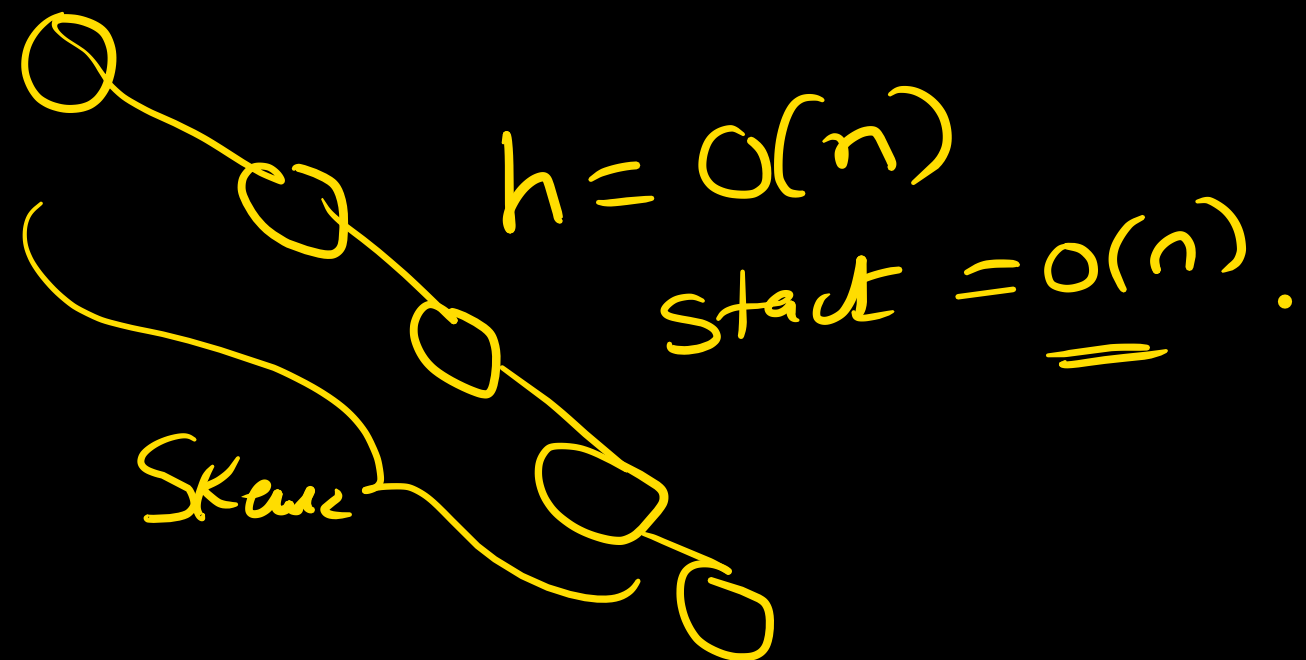
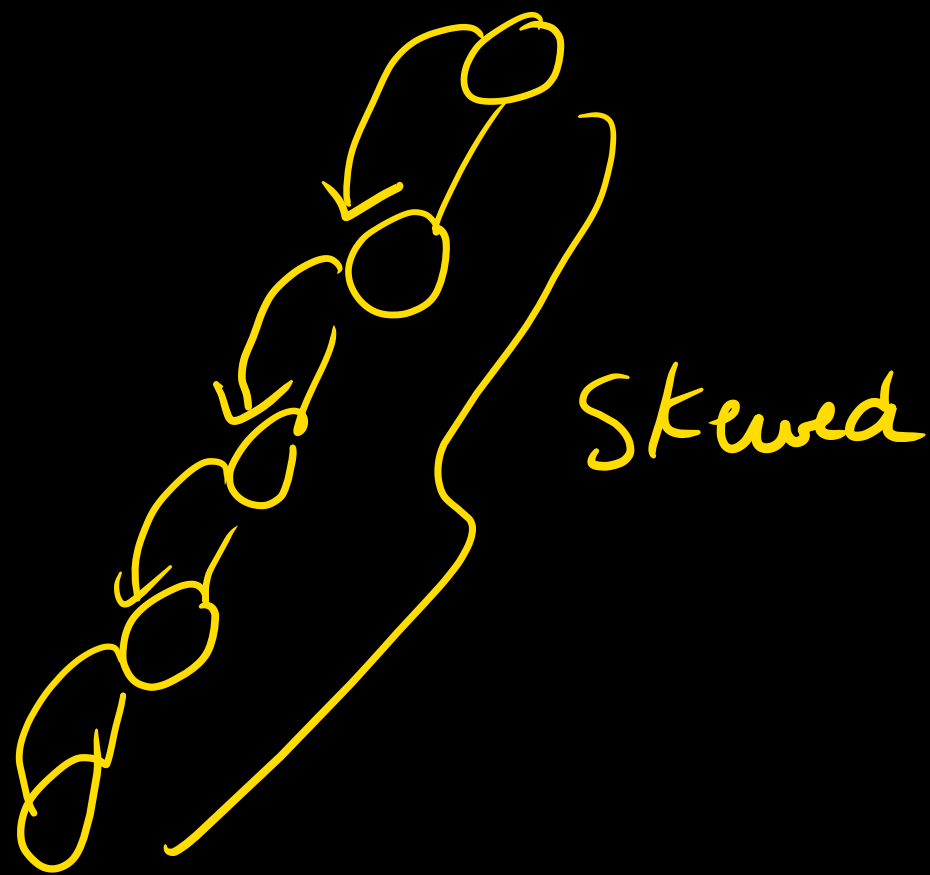
}

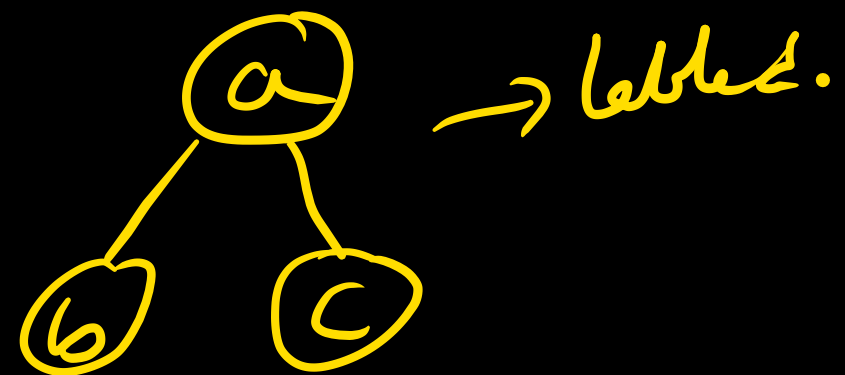
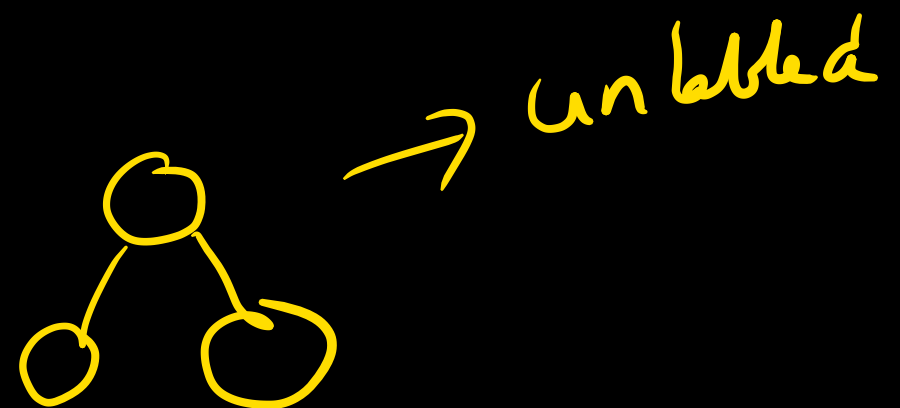
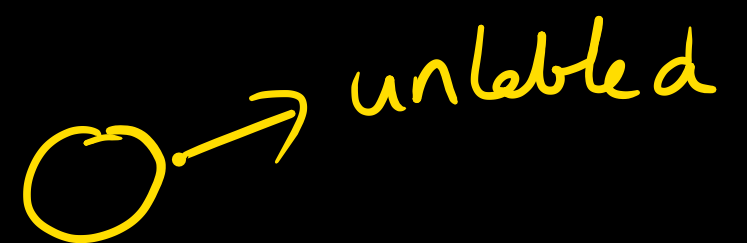
similar to
inorder.

A d B C . f g



	TC	SC
Pre	$O(n)$	$O(n)$ ✓
Inl	$O(n)$	$O(n)$ ✓
post	$O(n)$	$O(n)$ ✓





number of unlabeled binary trees:

Gate 2007

- III year

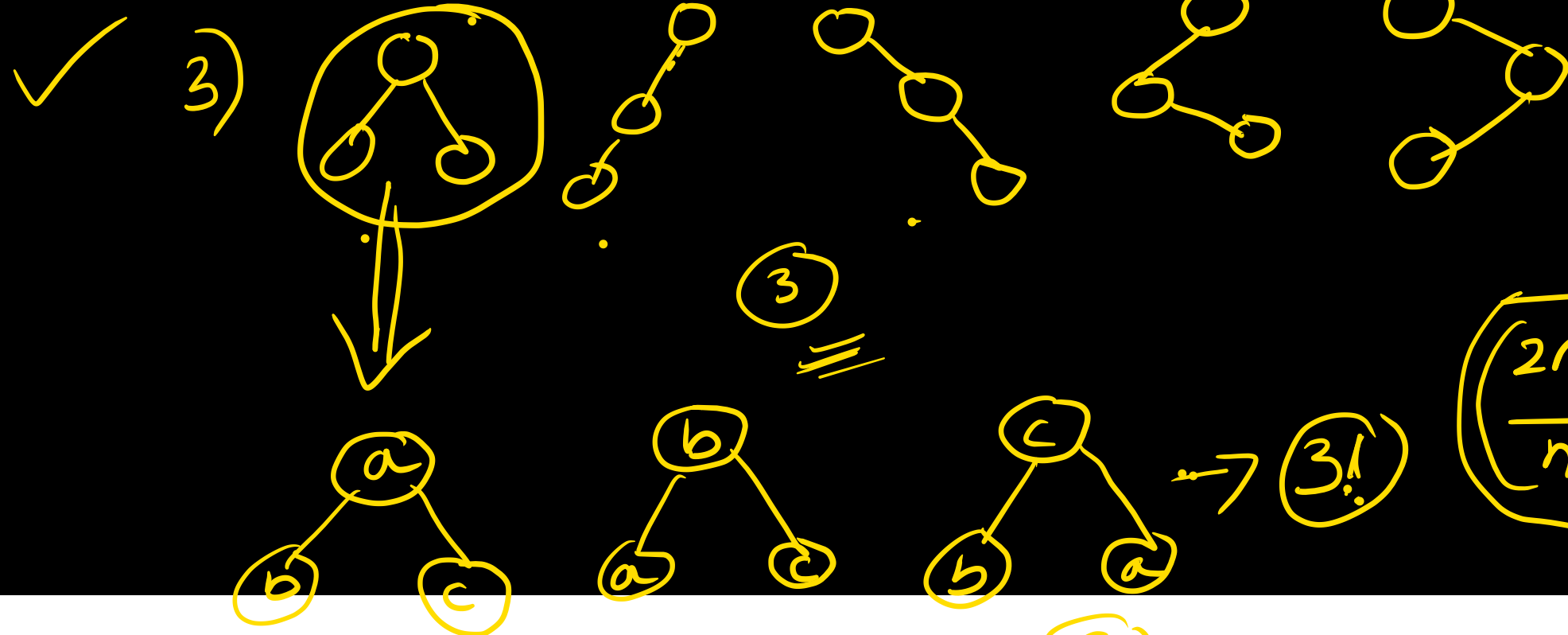
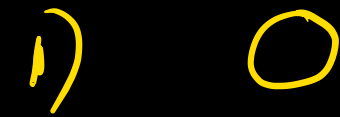
Btech.

IV

- 13

8

50



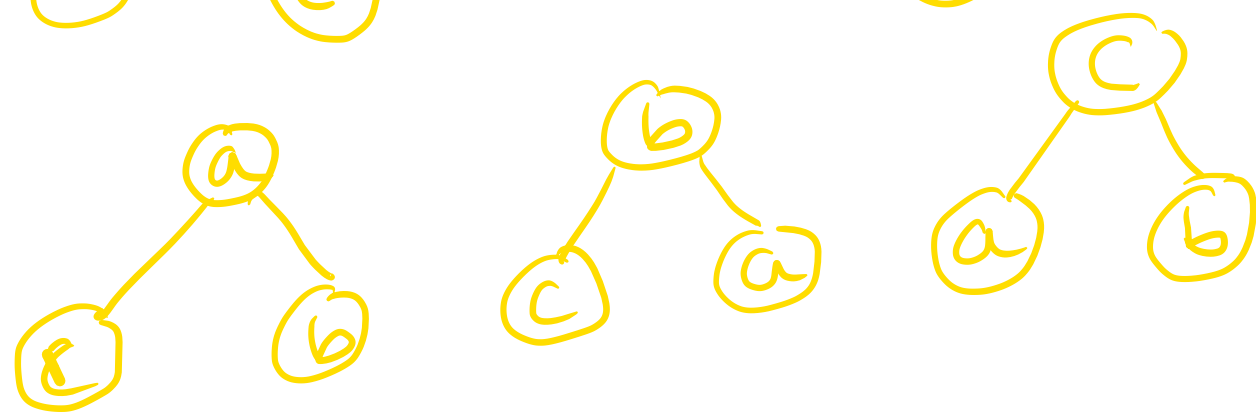
3

→ 3!

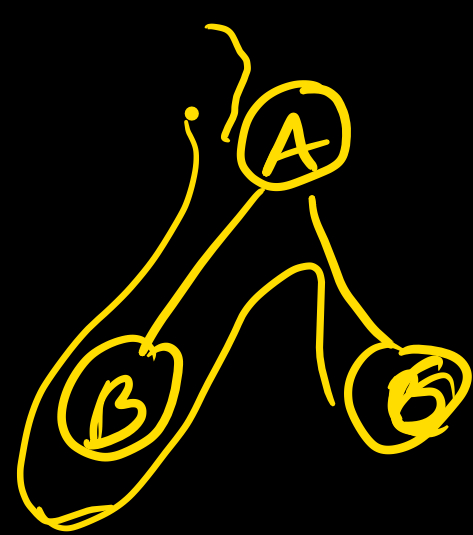
$$\left(\frac{2n(n)}{n+1} \right)$$

labeled =

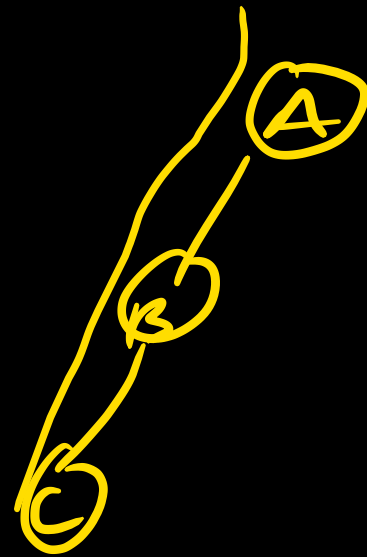
$$\left(\frac{2n(n)}{n+1} \right) \times n!$$



Construct a BT Given a pre order $\rightarrow$ uniquely



ABC

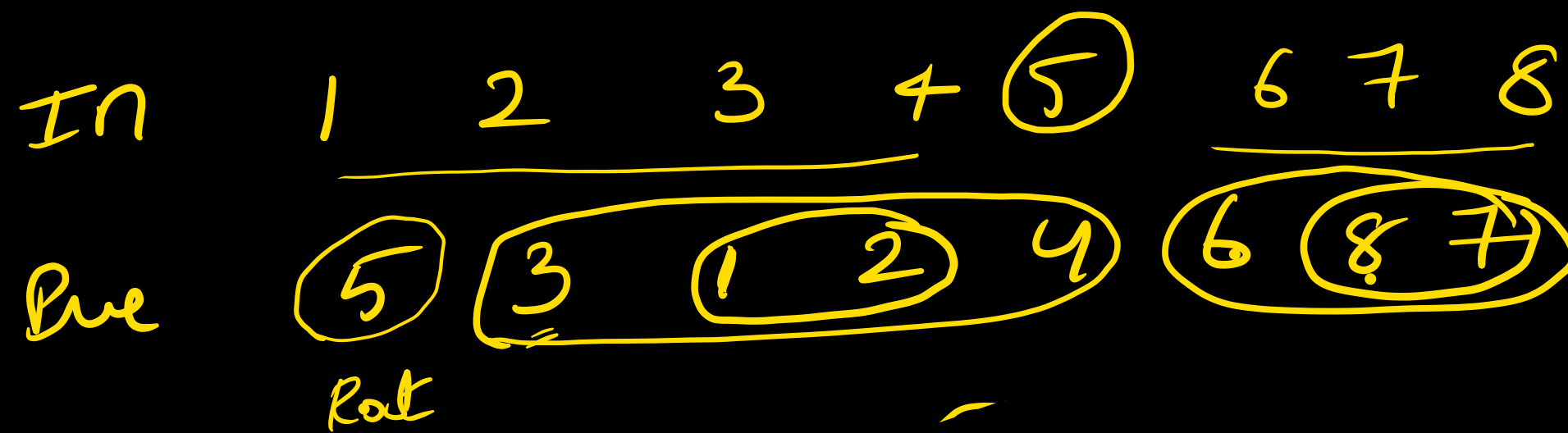


ABC.

Given a pre order we cannot construct a BT uniquely
" " "
" " "
" " "
" " "

Given In and pre
 In^d and $post$ } $\Rightarrow$ uniquely.

Given pre and $post$ } $\Rightarrow$ we cannot construct
a BT uniquely.



In: Left Root Right
Pre: Root left Right

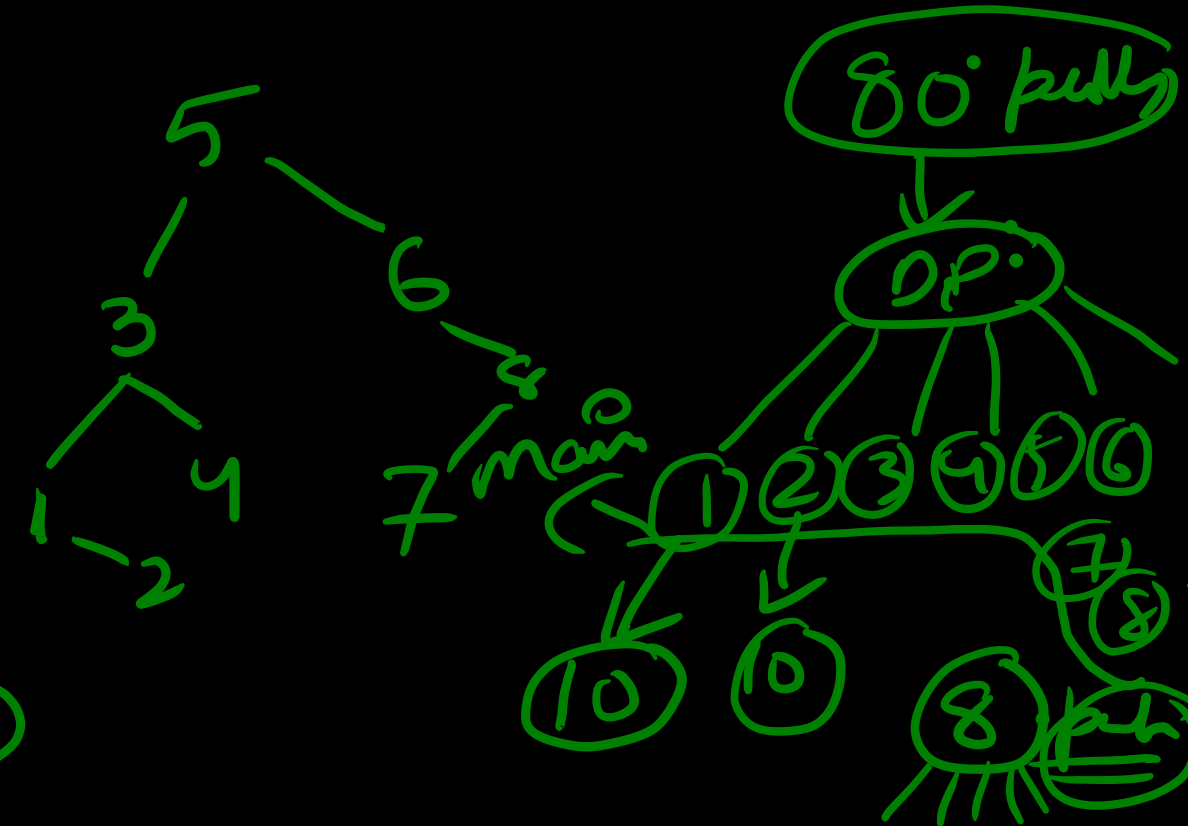
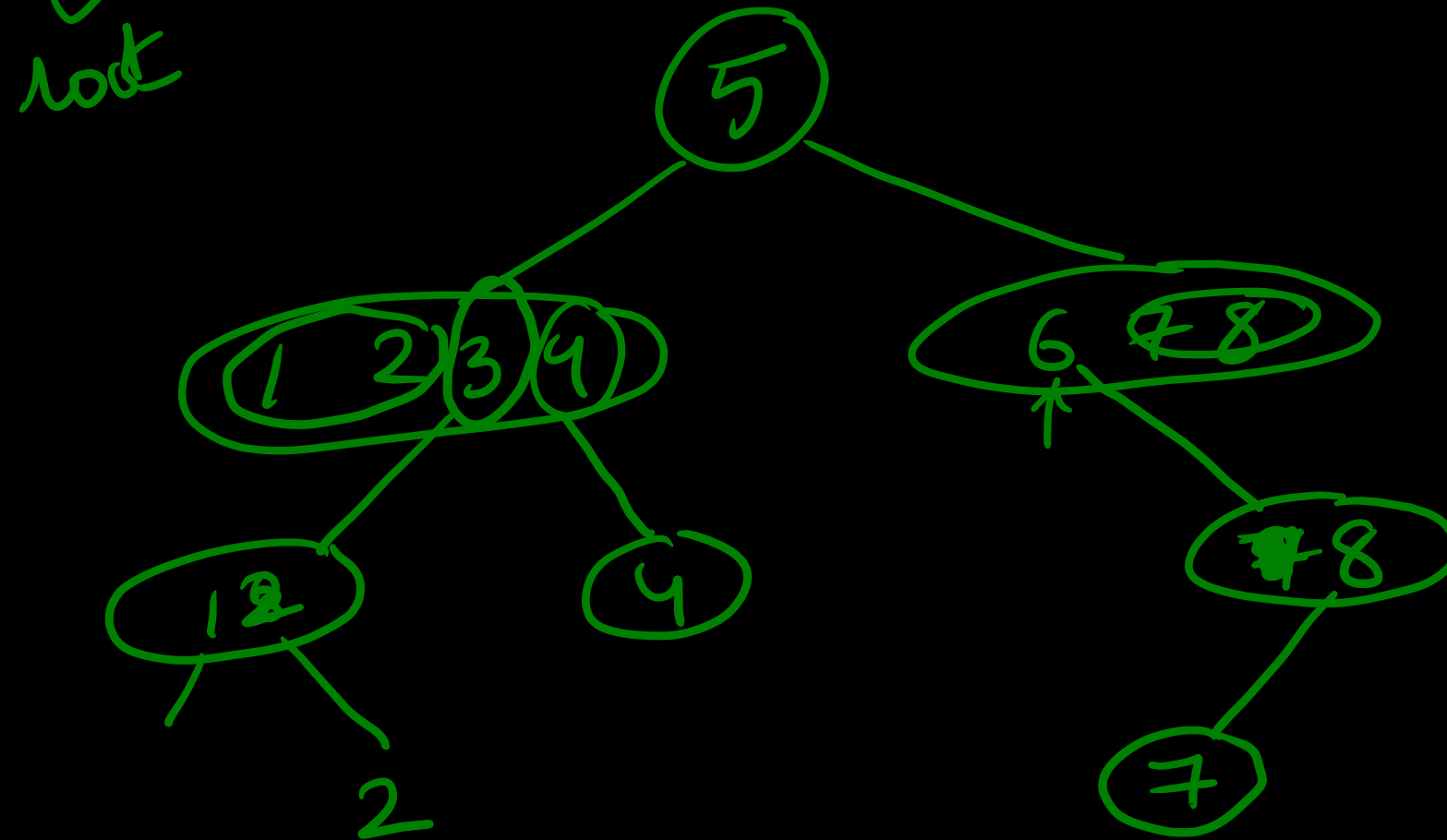
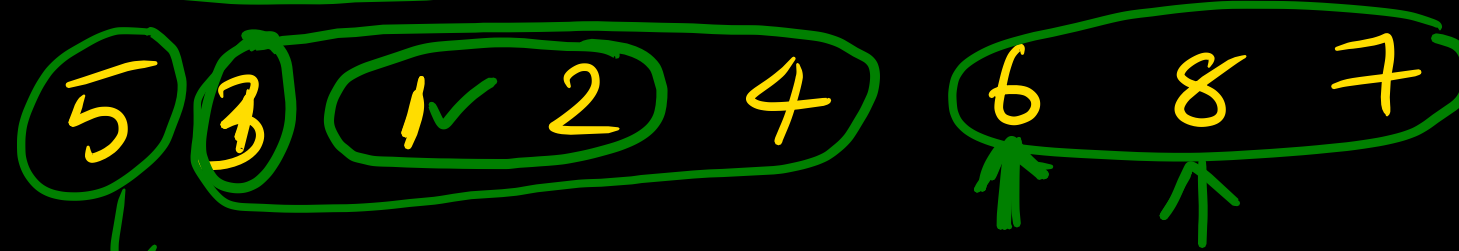
In

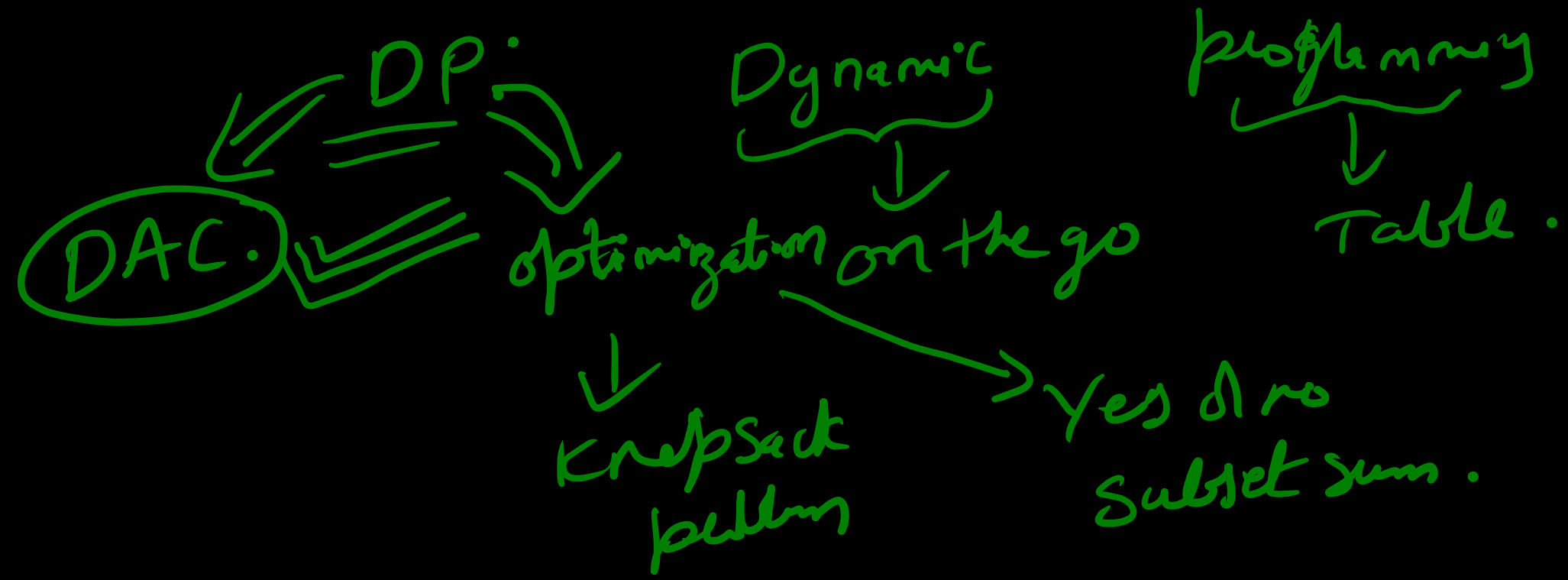


In: Left Root Right.

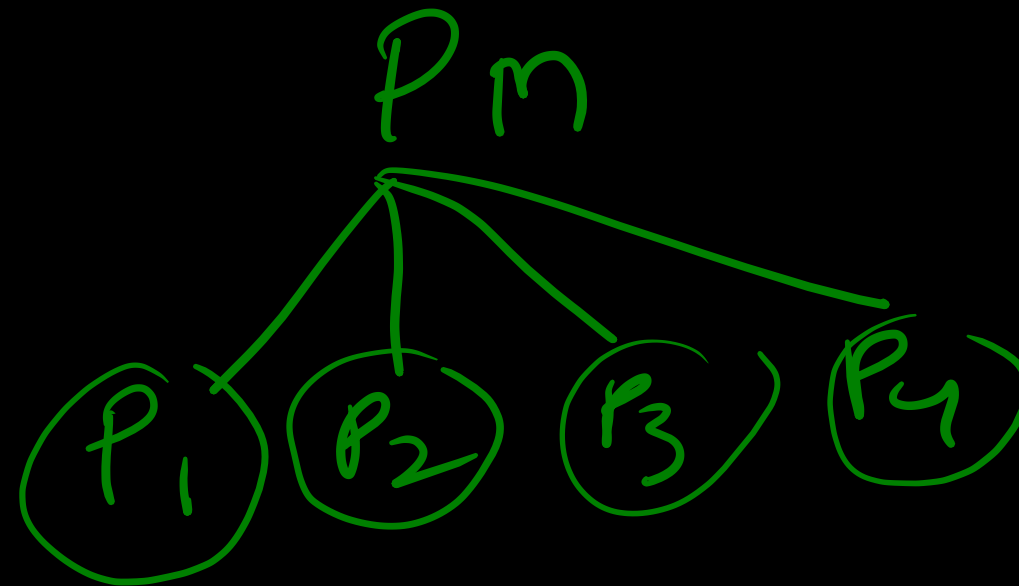
PRE: Root Left Right.

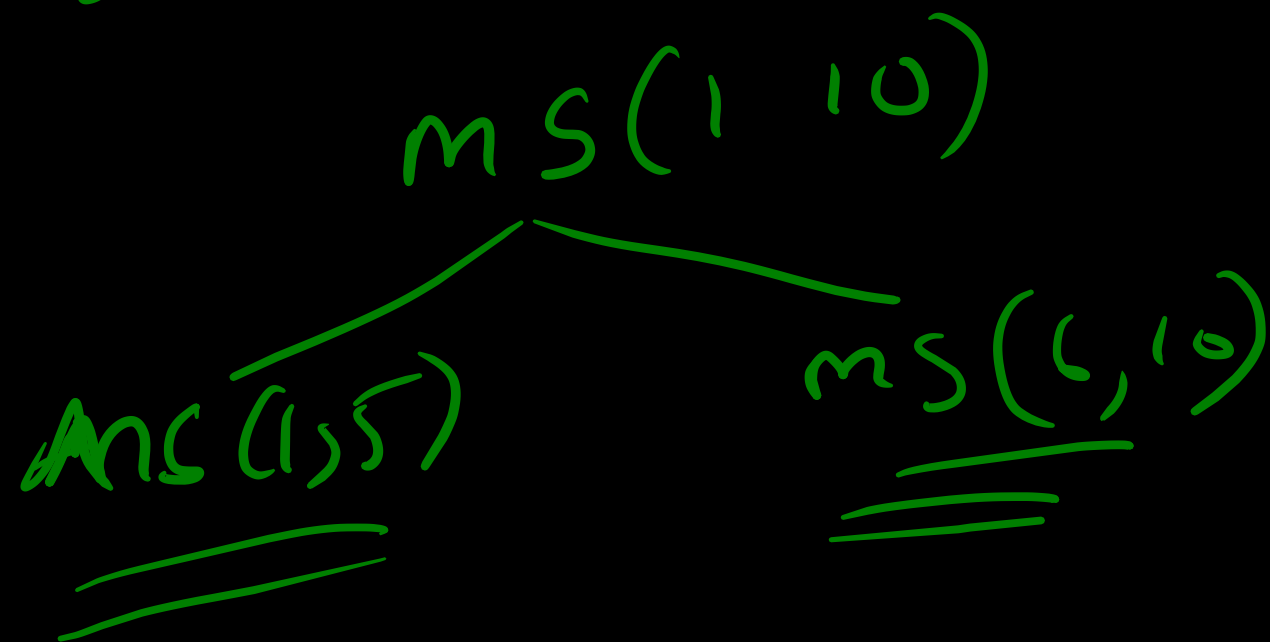
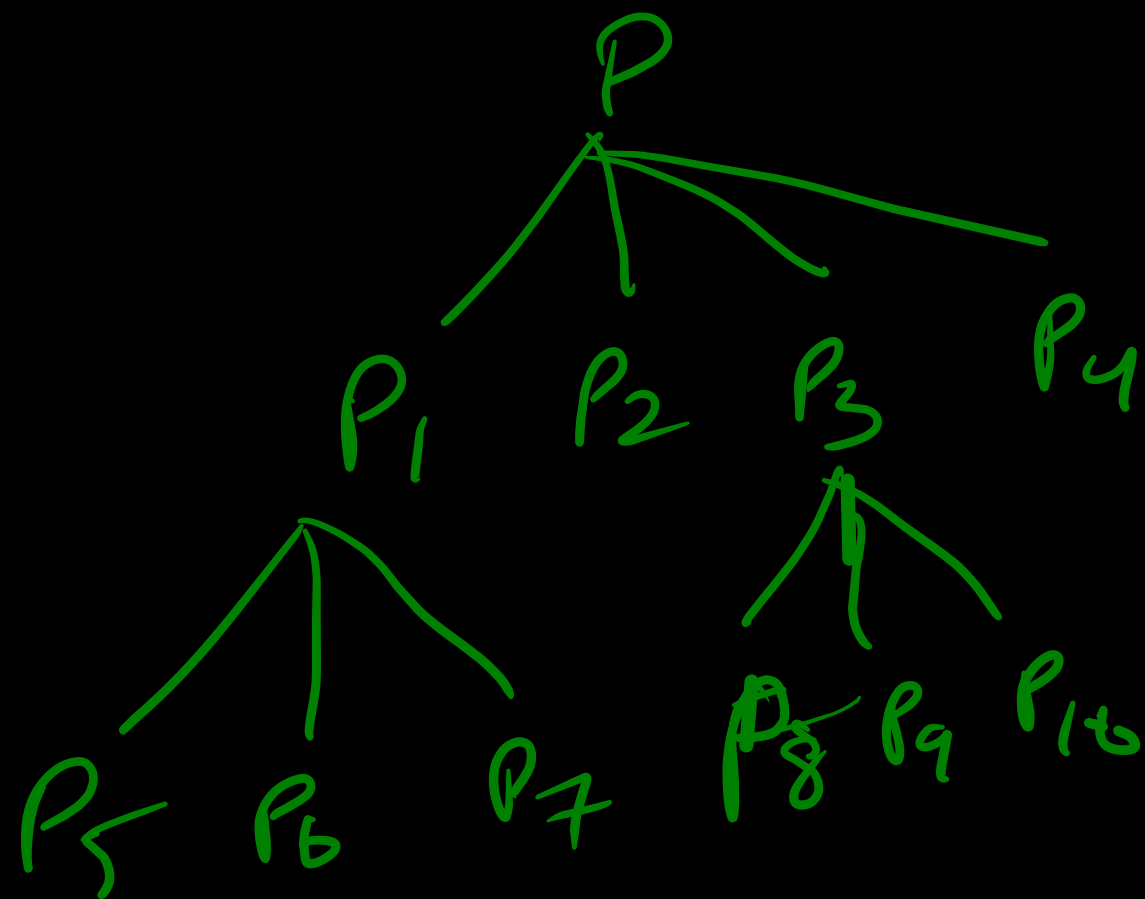
Pre





DAC

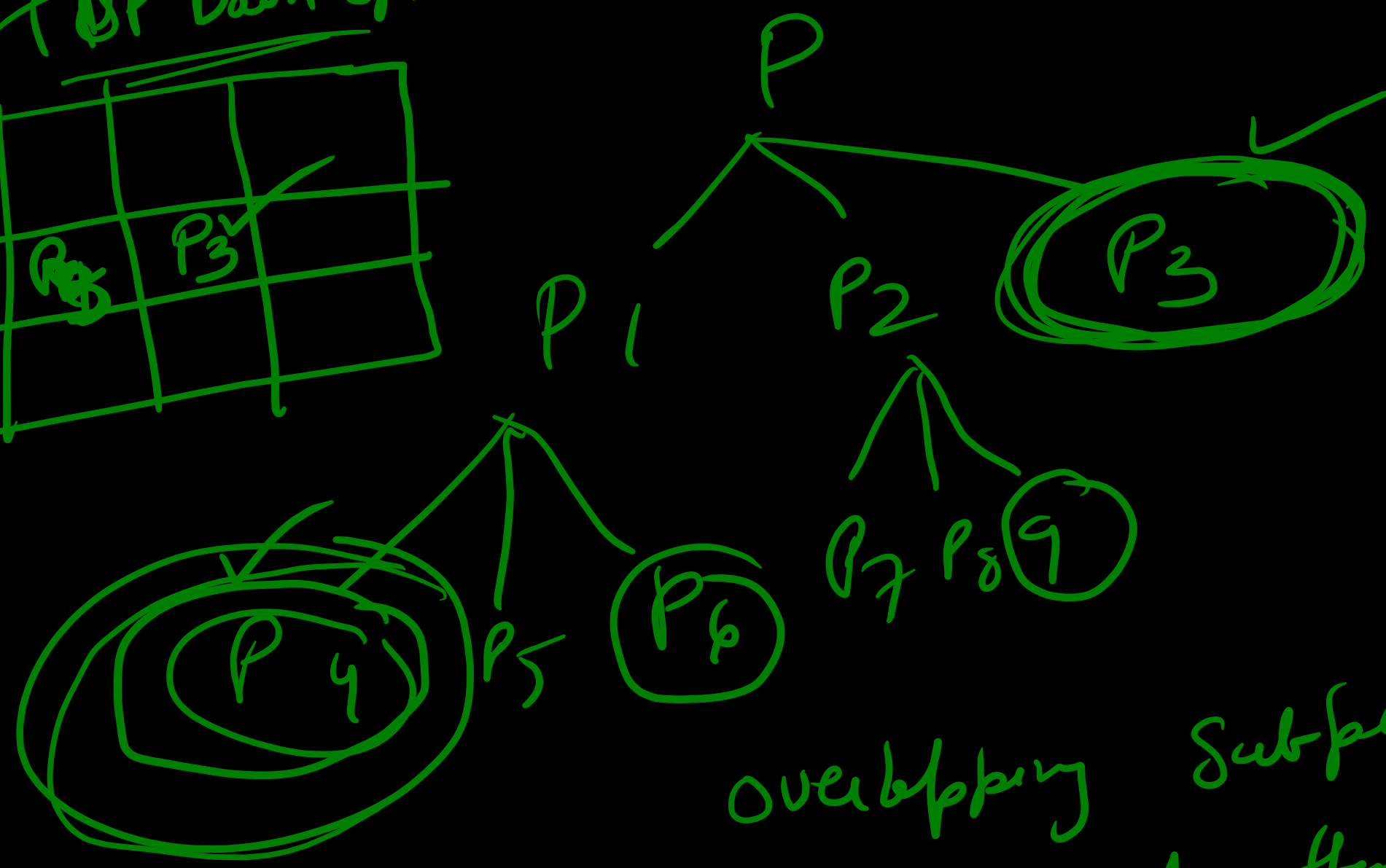
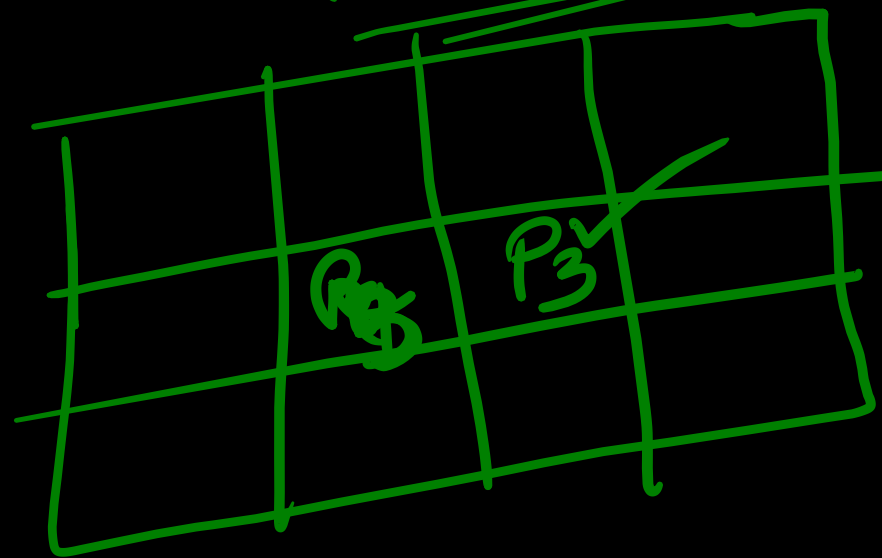




DAC

all the problem
are unique.

TSP Diagram



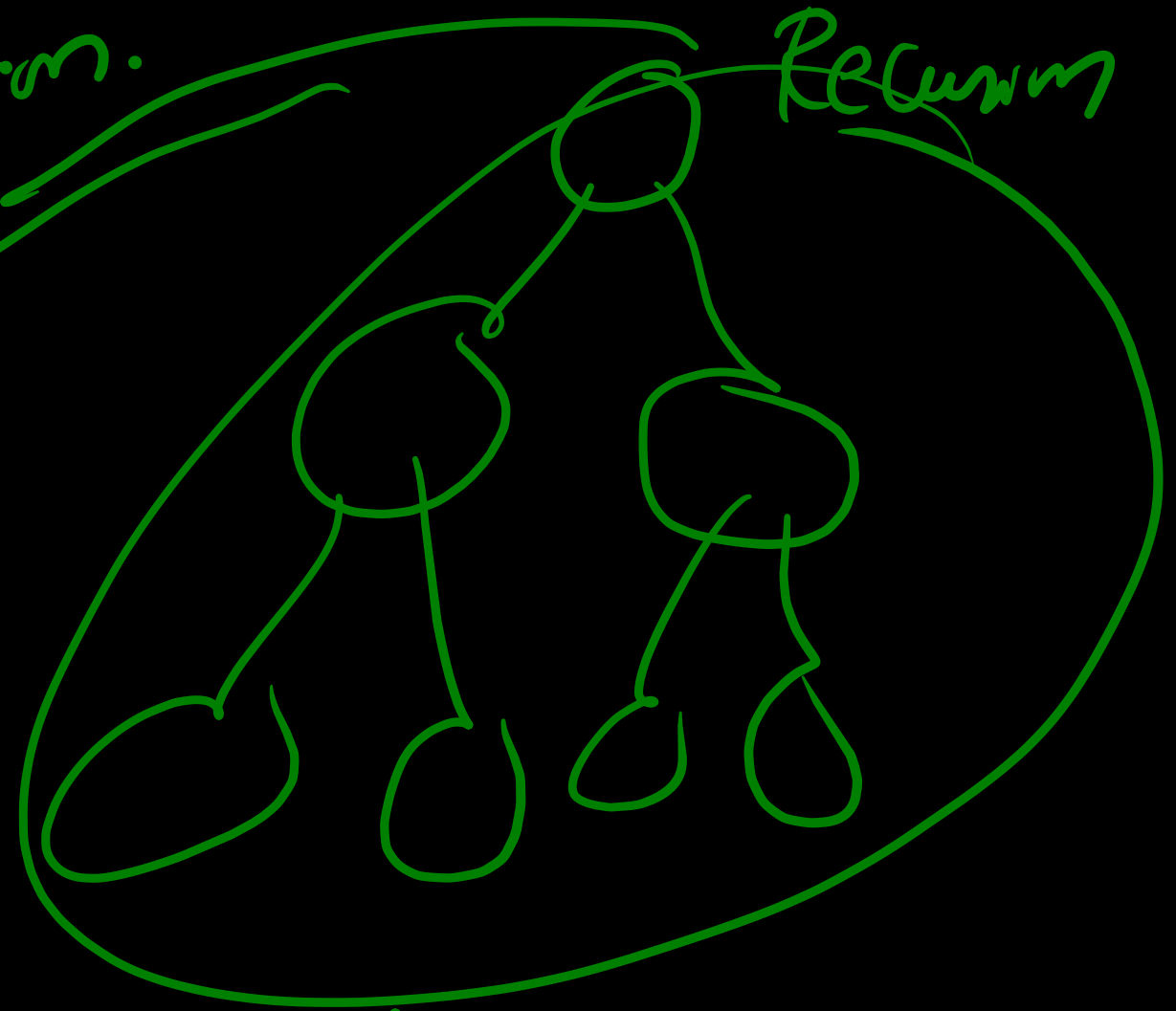
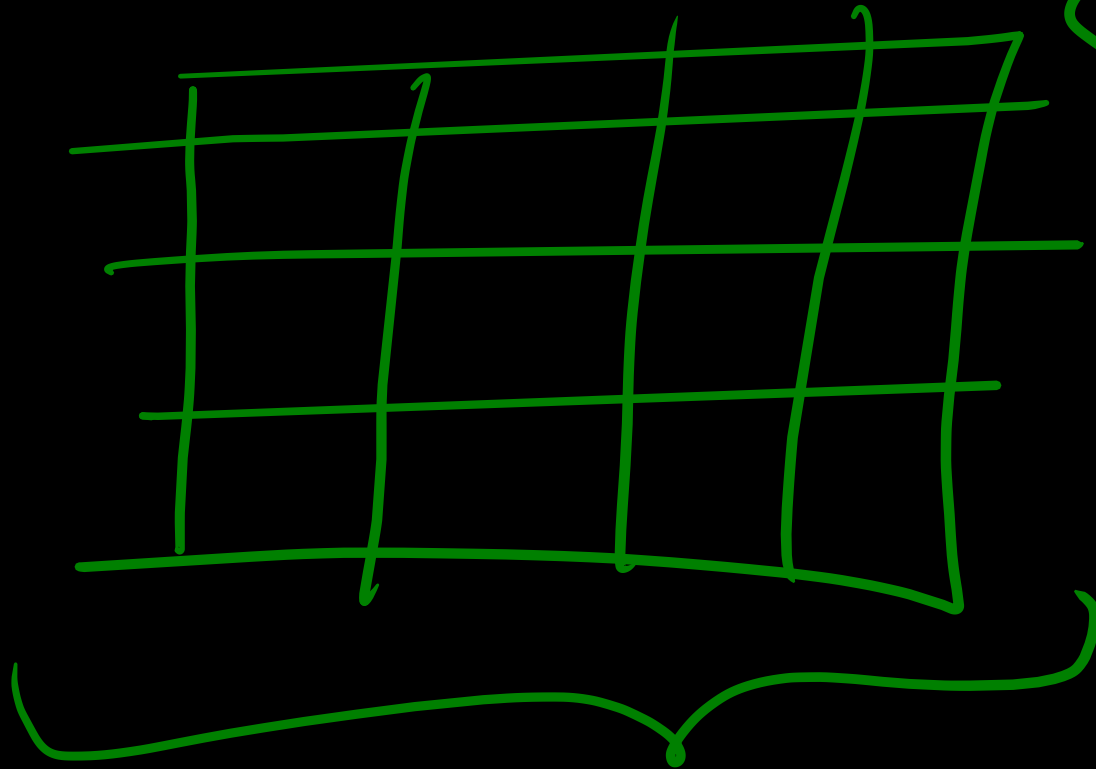
$O(n!)$ ✓

$O(2^n)$ ✓

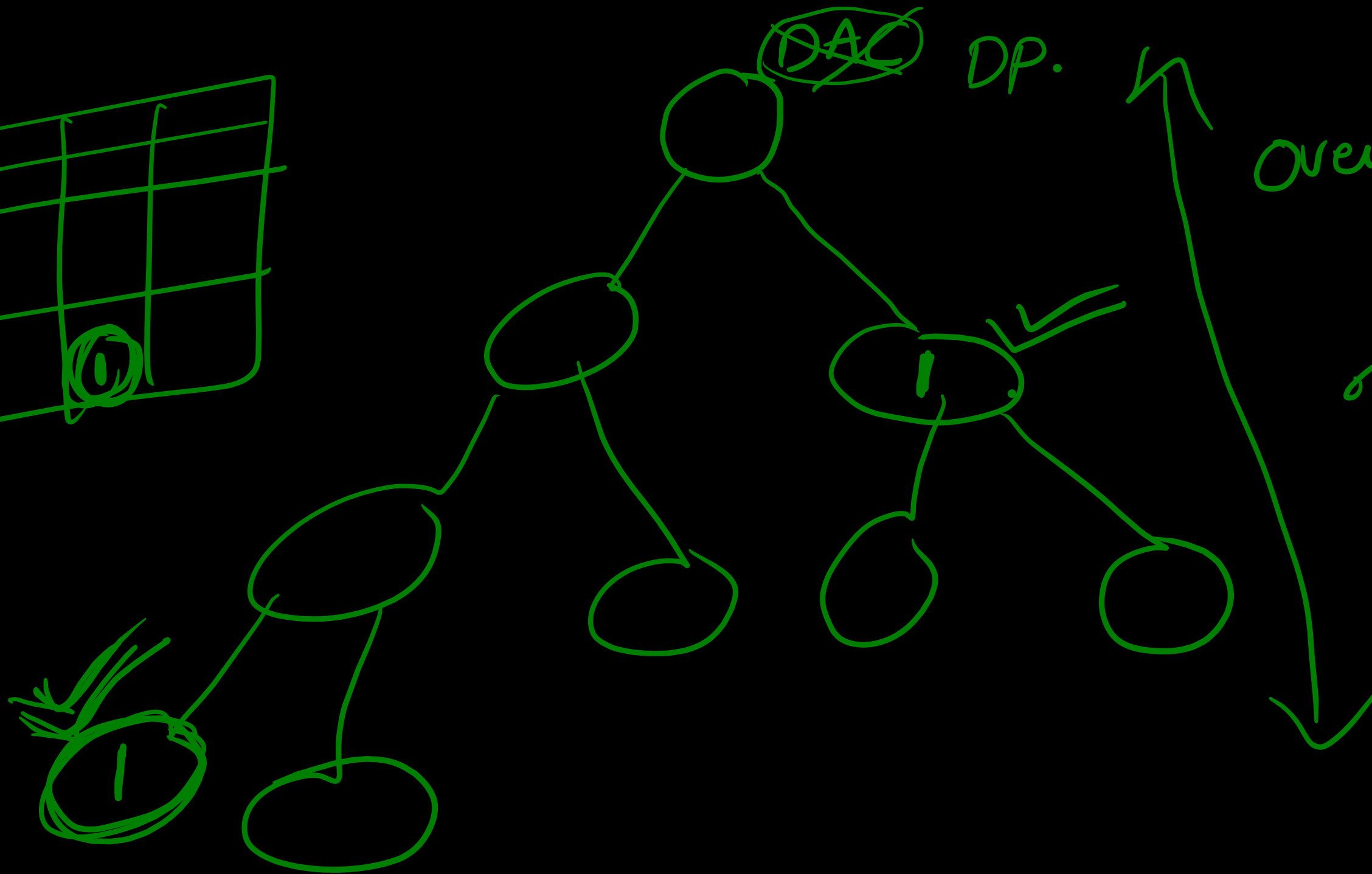
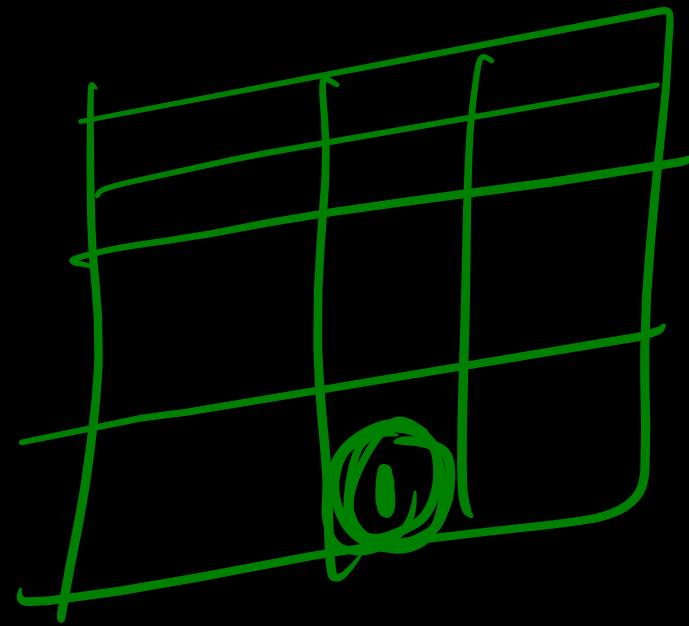
overlapping Subproblems.
we will not solve them
again

Bottom up ci.

iteration.



Recursive eqution $\Rightarrow$ table.



overlapping.

$O(2^n)$

→ ~~(Code)~~ Algo → who does this algo do.
} (they will say the algo → Fill in the blanks.)
able to write algo → Fill in the blanks
trace the algo and find out what it does.

① write algo. ✓

② trace them ✓

Gate:

formula

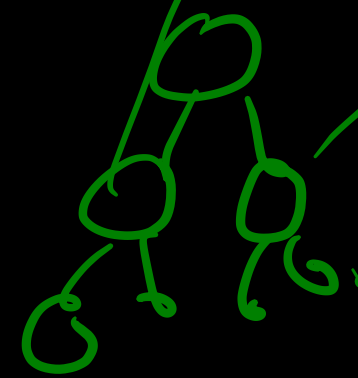
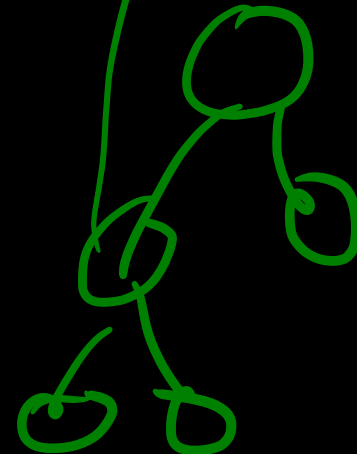
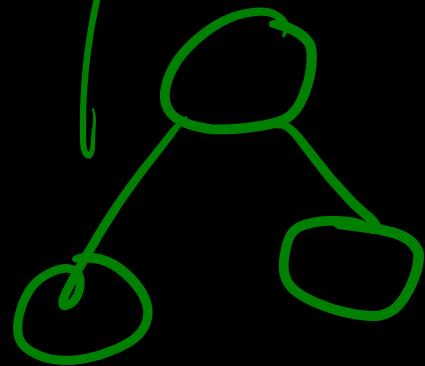
a) $n+1$

b) $n-1$

c) $\lceil n/2 \rceil$

d) $\lceil \sqrt{n} \rceil$

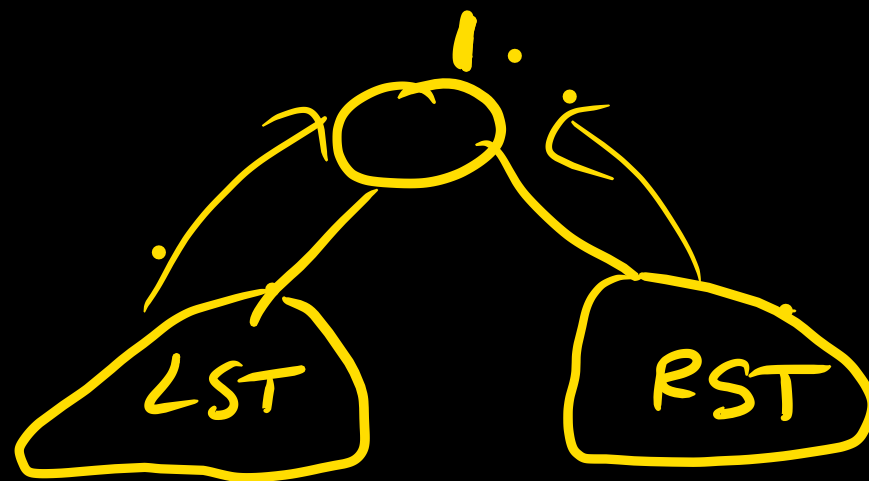
Don't derive formula.



write an algo to count no of nodes in a BT.

Recursive equations: $\rightarrow$ RA $\rightarrow$ Trac. 99% recursive ✓
1%

$$NN(T) = \begin{cases} 1 + \underline{\underline{NN(LST) + NN(RST)}}; & \text{if } T \neq \text{null} \\ 0; & \underline{\underline{T \text{ is null}}} \end{cases}$$




```
int NN(struct node *t)
```

Trace

```
{ if(t)
```

```
{ return 1 + NN(t->left) + NN(t->right));
```

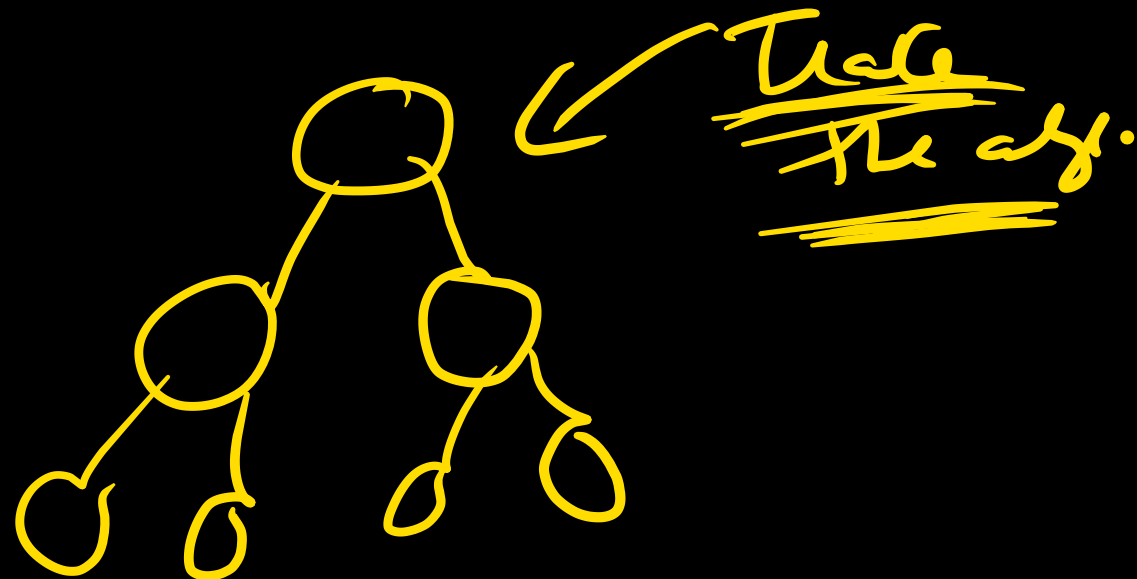
```
}
```

```
else
```

```
return 0
```

```
}
```

Random algo $\rightarrow$ what does it do



```
int NN(struct node *t)
```

```
{
```

```
if (t)
```

```
{ 1 L = NN(t → left)
```

```
2 R = NN(t → Right)
```

```
3 return (1 + L + R)
```

```
}
```

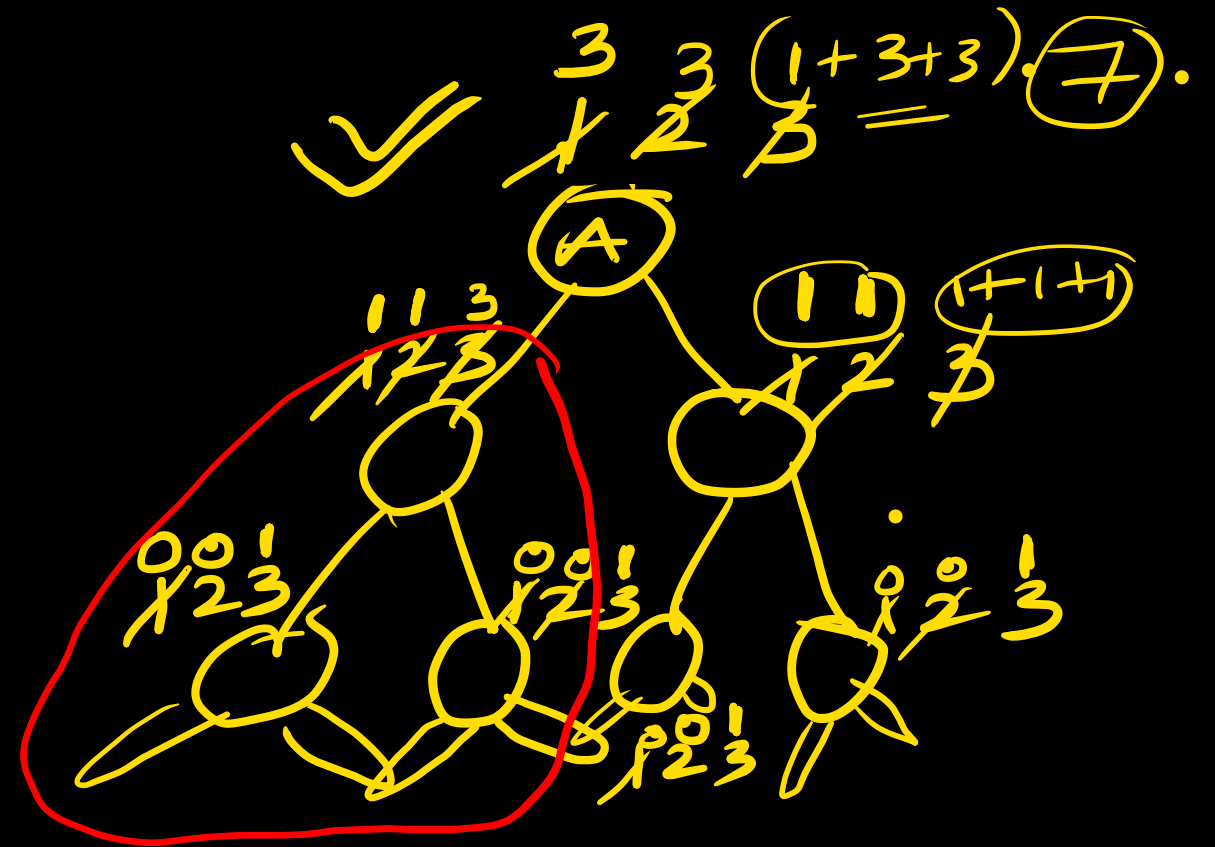
```
else  
return 0
```

```
}
```

Count the
nodes

Case

Non
leaf



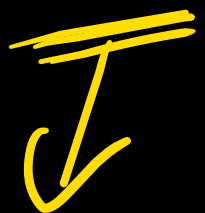
a) _____

b) _____

c) _____

_____ d) _____
to Count
the full tree.

RE



Recursive tree.
loop.

BT & Rec.

~~DP & Rec.~~
(over)

BT



ment for rec.

To write an algo to find the no of leaves.

$$NL(\text{struct node } *t)$$

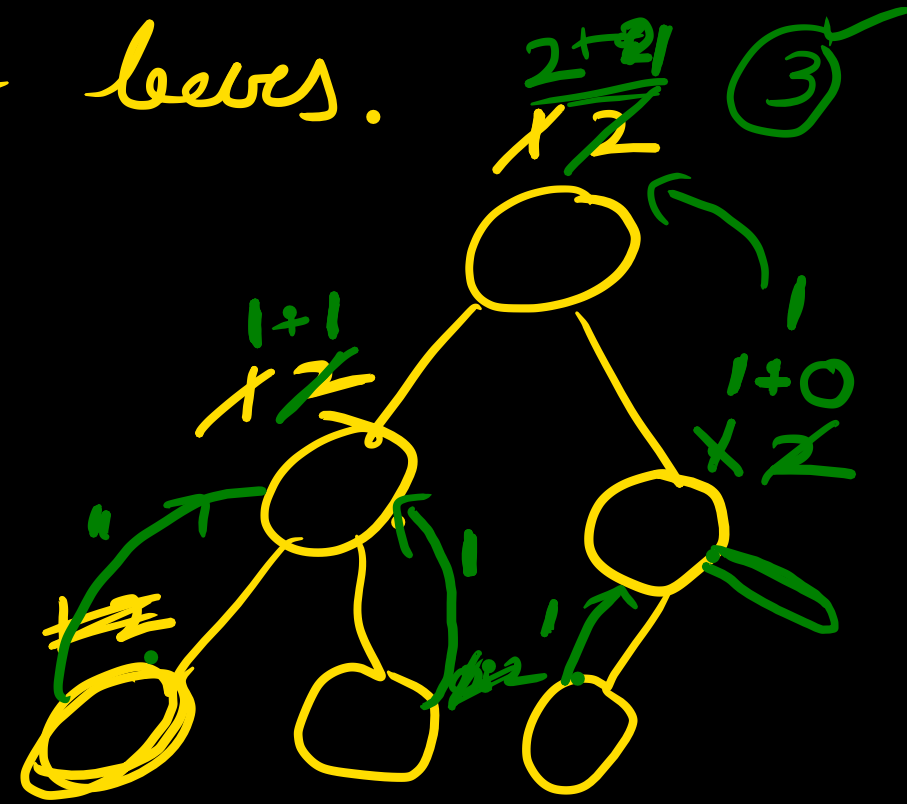
```
if (t == null) return 0;
```

if (t->left == null && t->right == null) ✓
return t; ✓

Return 1; ✓

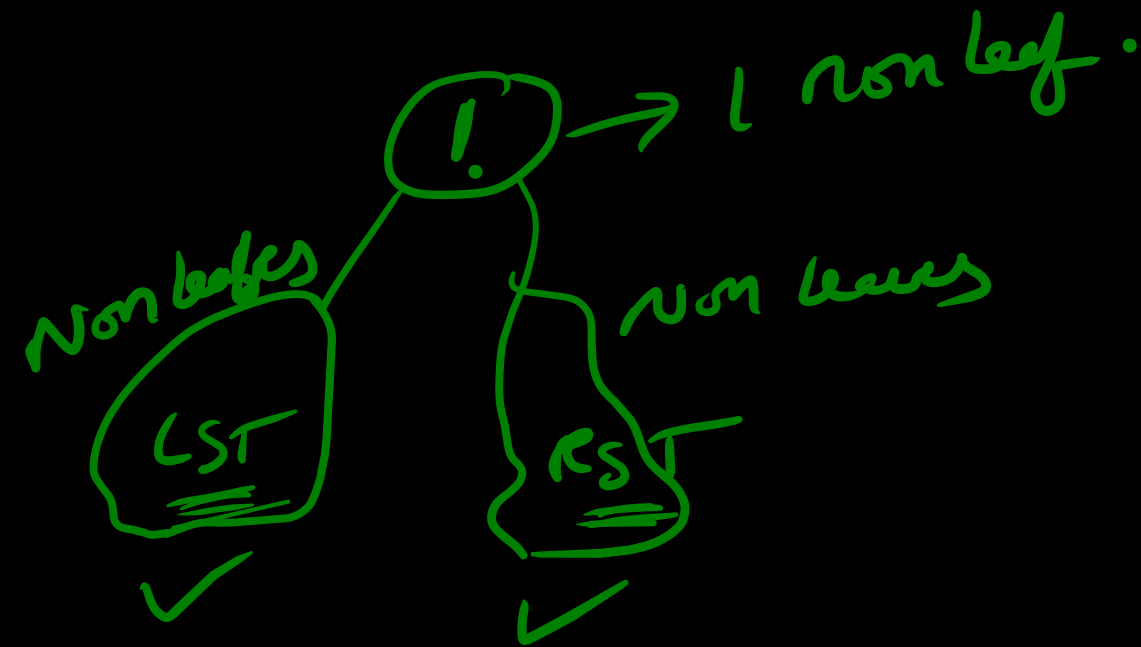
else return (NL(t-left) + NL(t-right));

3



Write an algo to find no of non leaves

$$NL(T) = \begin{cases} 0 & \text{if } T \text{ is a leaf.} \\ 1 + \underbrace{NL(LST)} + \underbrace{NL(RST)} & \text{otherwise.} \end{cases}$$

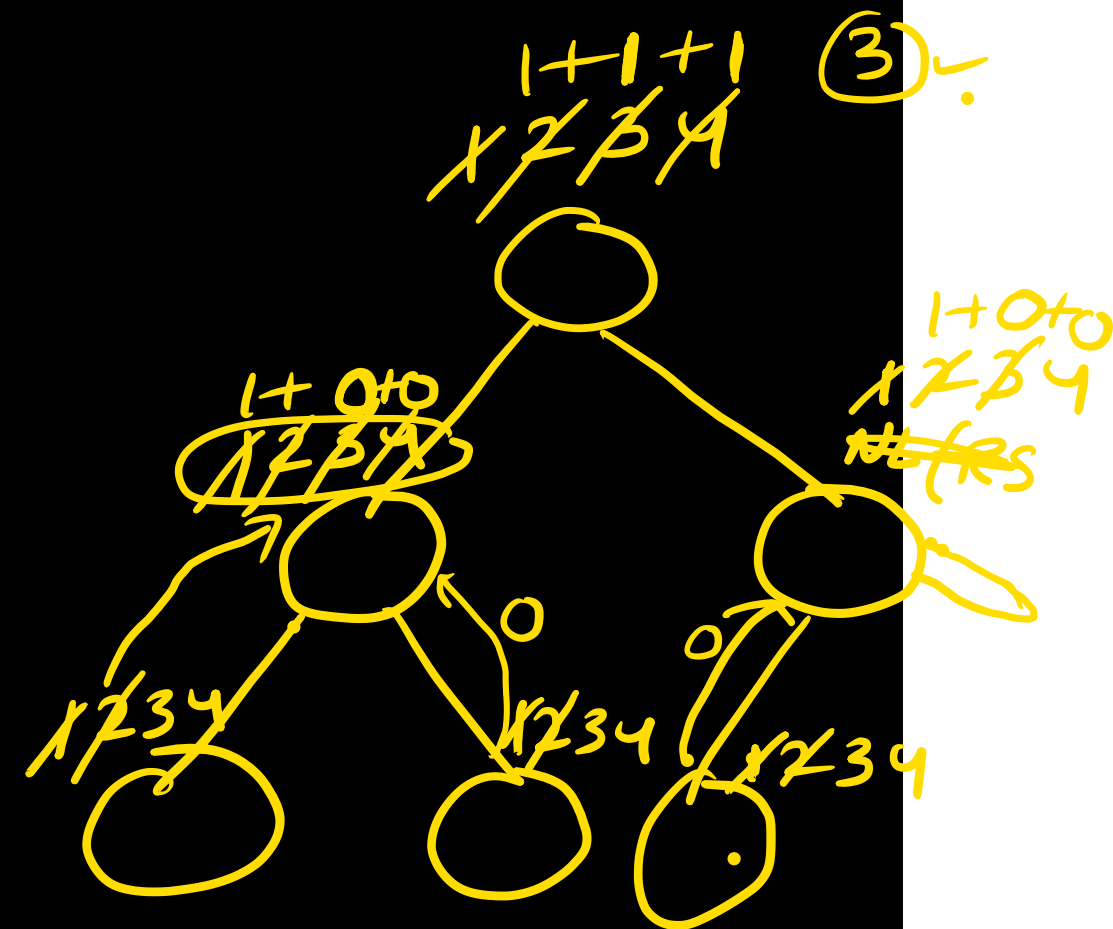


int NL(struct node *t)

{
① if (t == null) return 0;

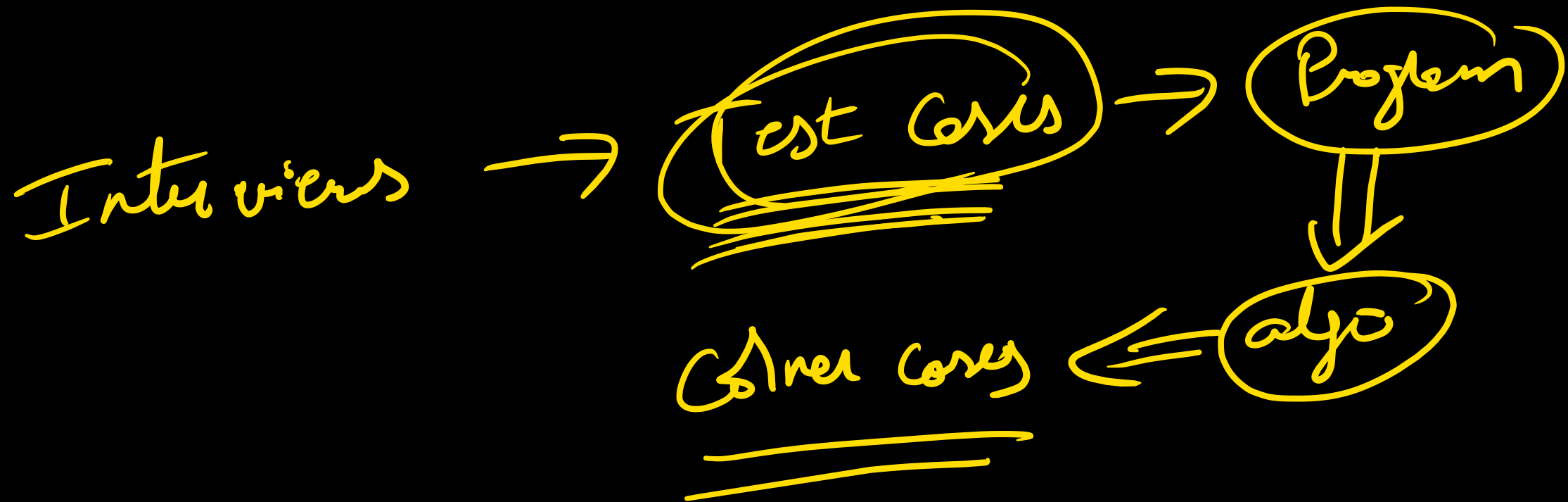
② if (t->left == null && t->right == null)
return 0;

else return (1 + NL(t->left) + NL(t->right))

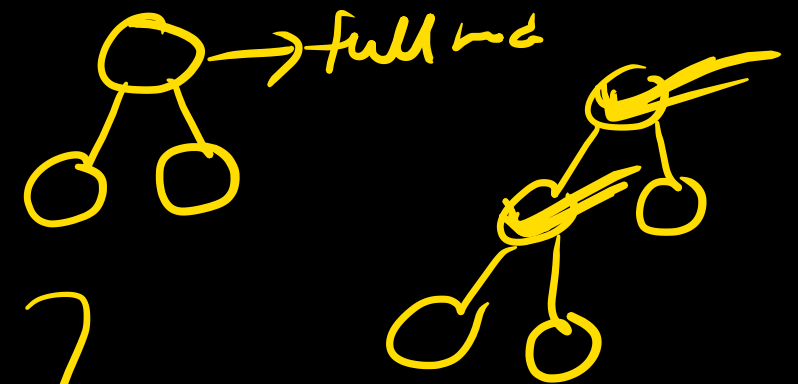


Q/w write the algo.

Corner cases and dry run.



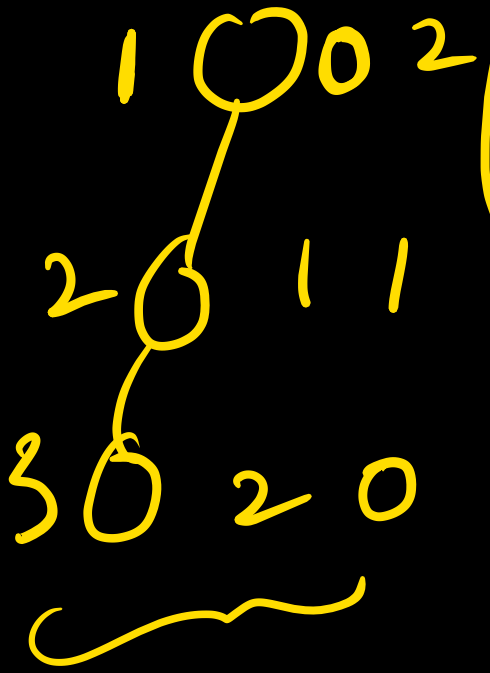
In a binary tree if a node has two children, then it is called full node.



level. { Complete tree $\leftrightarrow$ multiple definition
Full tree $\leftrightarrow$ multiple definition
Strict tree $\rightarrow$ multiple definition

Many books
has ~~many~~ many
def.

In the exam they will give you
def.

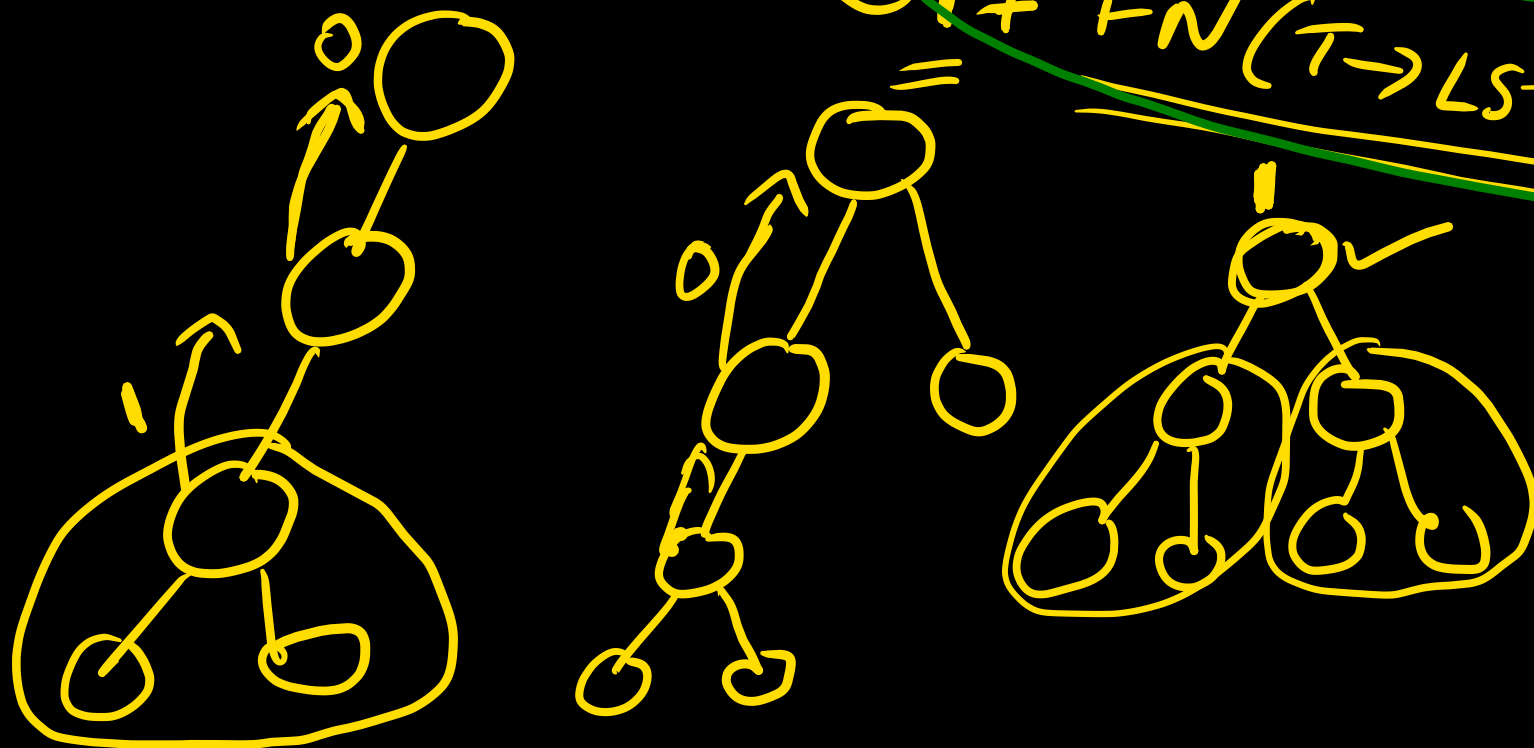


write an algorithm to find no of full nodes. $\rightarrow 0 \rightarrow 0$

$$FN(T) = \begin{cases} \underline{0} & ; T = \text{null} \\ 0 & ; T \text{ is a leaf} \end{cases}$$

$$0 + FN(T \rightarrow \underline{LST}) + FN(\underline{RST}) \quad \text{if } T \text{ has only one child}$$

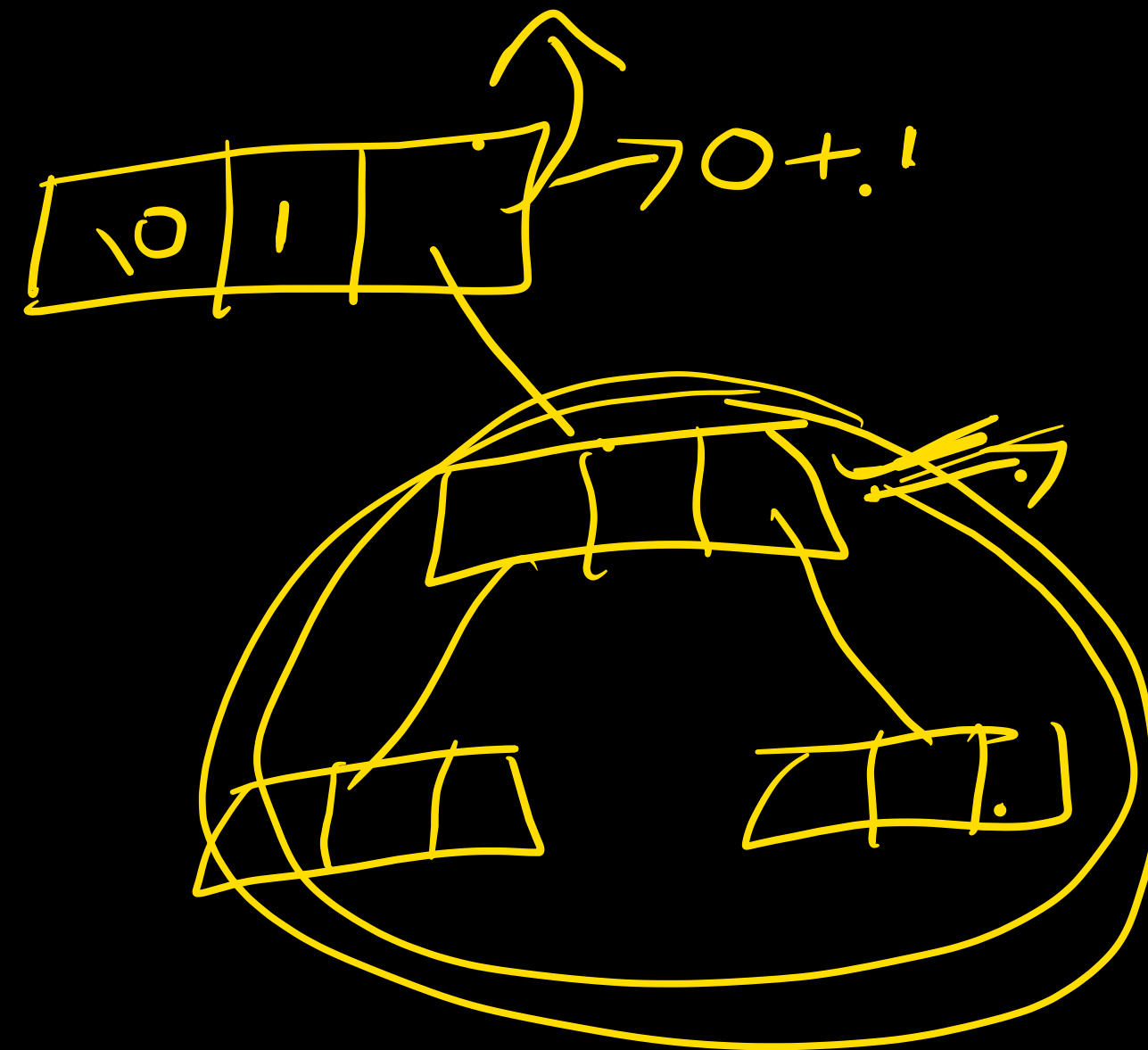
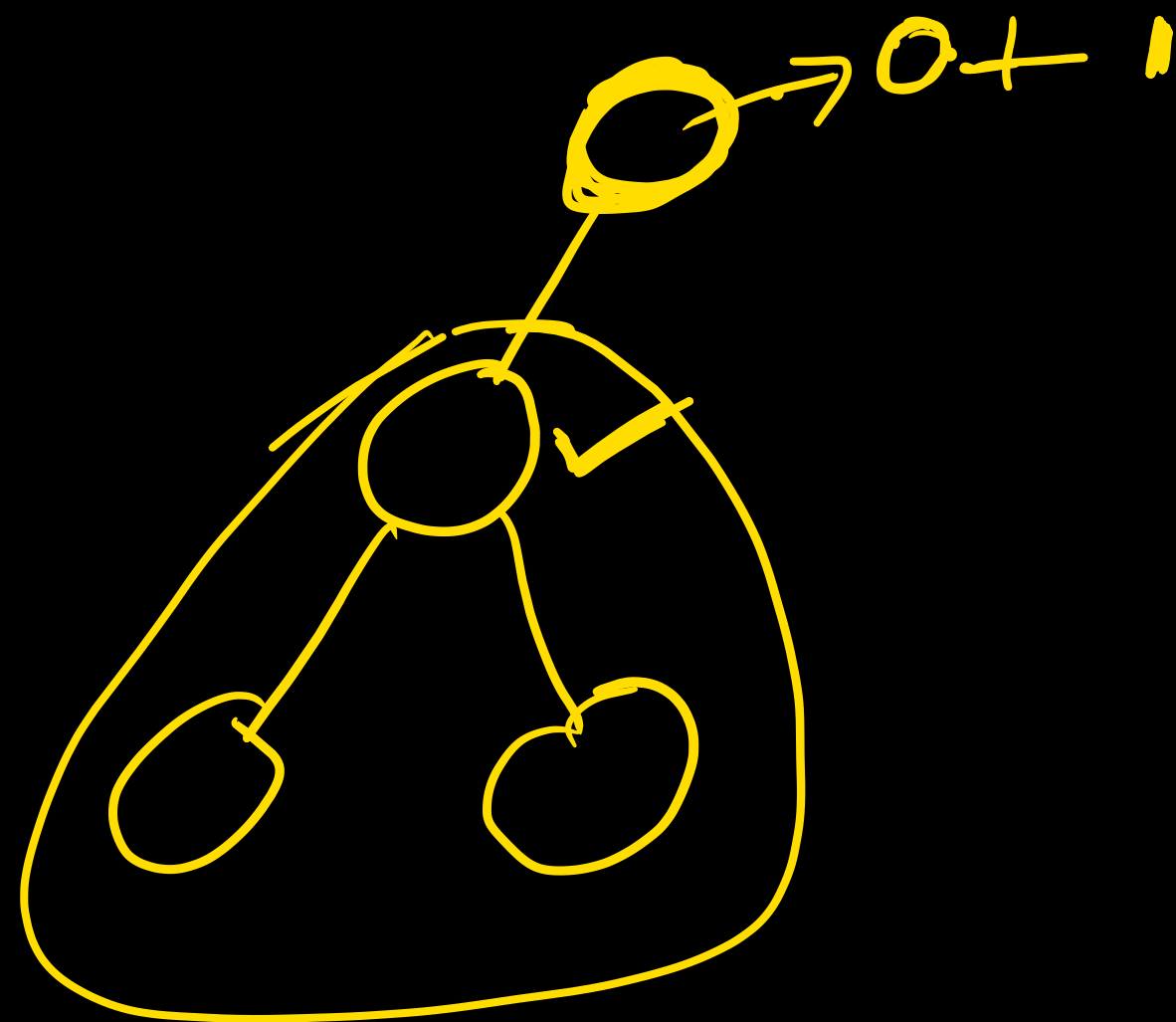
$$1 + FN(T \rightarrow LST) + FN(T \rightarrow RST) \quad \text{if } T \text{ has both children}$$



$$\boxed{\begin{matrix} \backslash & 0 & / \\ 0 & & 0 \end{matrix}} \Rightarrow \underline{0}$$

$$\boxed{\begin{matrix} \backslash & 0 & / \\ 0 & & \cdot \end{matrix}} \Rightarrow \begin{cases} 0 + FN(LST) \\ + FN(RST) \end{cases}$$

$$\boxed{\begin{matrix} \backslash & & / \\ & & \end{matrix}} \Rightarrow 1 + FN(LST) + FN(RST)$$



```
int FN(Struct node *t)
```

```
{
```

```
1 if (!t) return 0;
```

```
2 if (!t->left && !t->right)
    return 0;
```

```
return (FN(t->left) + FN(t->right))
```

```
+ (t->left && t->right) ? 1 : 0;
```

